

ML Design: Creator–Brand Recommendation System (10M Creators)

Candidate: Andrés Carvajal **Goal:** Top- K creators aligned to brand intent & constraints

Problem Overview (Assumptions)

We have $\approx 10\text{M}$ creator profiles with (i) long-lived profile signals (bio/taxonomy), (ii) recent content (captions/hashtags, optional transcripts) and (iii) rolling engagement metrics (e.g., 30d windows). Given a brand request (free-text + optional hard constraints + soft preferences), return Top- K creators that are *contextually aligned* while respecting constraints and keeping latency low at 10M scale.

This design reflects our end-to-end prototype validated on a 300K-profile simulation (indexing, retrieval, reranking, graded relevance evaluation, latency measurement): retrieval sub-100ms; cross-encoder rerank for ~ 200 pairs at $\sim 200\text{--}300\text{ms}$ on GPU; Precision@10 ≈ 0.32 , NDCG@10 ≈ 0.27 on a labeled subset.

1) Data Representation and Storage

1.1 Creator representation (hybrid schema)

Each creator c is represented by:

- **Structured:** follower_count, engagement_rate_30d, avg_views_30d, geo (city/country), language, demographics (optional)
- **Semi-structured:** verticals, keywords, hashtags (aggregated from posts)
- **Unstructured text:** bio + recent captions + optional transcripts
- **Trust/safety:** content_safety_labels, suspected_spam / authenticity score

Primary key: creator_id (stable for dedup + incremental updates).

1.2 Recency-weighted “creator document”

To capture topic drift, we represent each creator by a single text document composed of stable profile signals and recency-weighted content.

Post-level recency weight.

Each post i receives an exponential decay weight based on its age:

$$w_i = \exp\left(-\frac{\Delta t_i}{\tau}\right)$$

where Δt_i is the post age and τ is a tunable half-life parameter.

Creator document construction.

The creator document is defined as the concatenation of three components:

$$\text{doc}(c) = \underbrace{\text{bio}}_{\text{profile}} \parallel \underbrace{\text{tax}(c)}_{\text{taxonomy}} \parallel \underbrace{\text{posts}(c)}_{\text{recency-weighted content}}$$

The recency-weighted post block is given by:

$$\text{posts}(c) = \text{Concat}_{i=1}^m \left(w_i \odot \text{text}(\text{post}_i) \right)$$

Here, $\parallel$ denotes token/text concatenation, and $\odot$ represents a practical weighting mechanism applied at embedding time

(e.g., templated emphasis, token repetition, or learned recency features), rather than literal scalar multiplication of strings.

1.3 Storage/indexing at 10M (ML-facing)

Store in **OpenSearch** using a hybrid index:

- **Lexical retrieval:** BM25 inverted index over $\text{doc}(c)$
- **Semantic retrieval:** dense vector field $\mathbf{e}_c \in \mathbb{R}^d$ for ANN (HNSW)
- **Filterable fields:** follower_count, geo, language, safety labels, spam/authenticity

Explicit libraries/tools (aligned to our prototype stack):

- Search/index: **opensearch-py** client; OpenSearch k-NN plugin (HNSW)
- Embeddings: **sentence-transformers**
- Reranking: **transformers** + **torch** (batched GPU inference)
- Evaluation: **numpy**, **pandas** (Precision/NDCG utilities)

2) Similarity Definition and Measurement

2.1 Score decomposition

Relevance is (semantic alignment) + (business fit):

$$S(c, q) = \alpha S_{\text{sem}}(c, q) + \beta S_{\text{perf}}(c) + \gamma S_{\text{pref}}(c, q) - \delta S_{\text{risk}}(c)$$

where $\alpha, \beta, \gamma, \delta$ are tunable hyperparameters calibrated via validation.

- S_{sem} : learned semantic relevance (dense retrieval + cross-encoder rerank)
- S_{perf} : normalized performance priors (engagement/reach)
- S_{pref} : soft preference match (geo/lang/vertical affinity)
- S_{risk} : safety/spam penalties (optionally brand-specific)

Hard constraints (e.g., min_followers) are enforced as filters before scoring; free-text constraints are parsed into structured filters.

2.2 Concrete definitions (math)

Bi-encoder similarity:

$$\mathbf{e}_q = f_{\text{bi}}(q), \quad \mathbf{e}_c = f_{\text{bi}}(\text{doc}(c)), \quad s_{\text{bi}}(c, q) = \cos(\mathbf{e}_q, \mathbf{e}_c)$$

Cross-encoder relevance (interaction model):

$$s_{\text{ce}}(c, q) = f_{\text{ce}}(q \parallel \text{doc}(c))$$

where $\parallel$ denotes token-level concatenation in the model input.

Performance prior example (robust, scale-friendly):

$$S_{\text{perf}}(c) = \lambda_1 z(\log(1 + \text{avg_views}_{30d})) + \lambda_2 z(\text{engagement_rate}_{30d})$$

with robust standardization

$$z(x) = \frac{x - \text{median}}{\text{IQR} + \epsilon}$$

(optionally computed per vertical/geo slice), and tunable λ_i .

Risk penalty (simple default):

$$S_{\text{risk}}(c) = \eta_1 \cdot \mathbb{I}\{\text{suspected_spam}\} + \eta_2 \cdot \mathbb{I}\{\text{safety_label} \in \mathcal{B}\}$$

where $\mathcal{B}$ is a blocked/penalized label set (brand-dependent), and η_i are tunable.

3) System Architecture + Model

3.1 Multi-stage ranking pipeline

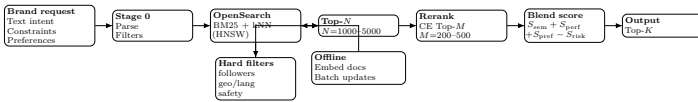
1. **Stage 0 (request understanding)**: parse q into query text + hard constraints + soft preferences.
2. **Stage 1 (candidate generation)**: hybrid retrieval in OpenSearch (BM25 + ANN), apply hard filters, return Top- N candidates ($N \approx 1000-5000$).
3. **Stage 2 (precision rerank)**: cross-encoder rerank Top- M ($M \approx 200-500$), then blend with priors to output Top- K .

3.2 Explicit model choices

Defaults that match speed/quality tradeoffs:

- **Bi-encoder (embeddings)**: sentence-transformers all-MiniLM-L6-v2
- **Bi-encoder (multilingual)**: sentence-transformers paraphrase-multilingual-MiniLM-L12-v2
- **Cross-encoder (reranker)**: MiniLM cross-encoder cross-encoder/ms-marco-MiniLM-L-6-v2

3.3 Architecture diagram (compact)



4) Hard Cases / Expected Failure Modes

- **Sparse or ambiguous creators**: little text but strong engagement signals $\Rightarrow$ semantic retrievers under-rank. Mitigation: gate performance boosts behind a minimum semantic threshold.
- **Spammy keyword stuffing**: creators inject hashtags to match queries. Mitigation: cross-encoder interaction scoring + authenticity/risk features reduce false positives.
- **Multilingual/code-switching**: English brand query vs Spanish-heavy creators. Mitigation: multilingual bi-encoder or lightweight translation; language-aware retrieval + filters.
- **Topic drift**: creator content changes; wrong τ under/over-reacts. Mitigation: tune τ by vertical; optionally maintain both long-term and recent documents.

5) Latency at 10M Scale (Without Sacrificing Quality)

Key principle: **retrieve wide, rerank narrow**.

- ANN (HNSW) reduces 10M to Top- N quickly; BM25 complements with exact-term recall.
- Cross-encoder is the expensive step: apply only to Top- M with GPU batching.
- Use a dynamic budget: increase M for ambiguous queries (multi-intent), reduce M for simple queries.

Observed in our simulation: retrieval sub-100ms; CE rerank for ~ 200 pairs at $\sim 200-300$ ms on GPU. Critically, runtime scales with M (candidate depth), not corpus size.

6) Bottlenecks / Key Design Decision Points

- **Representation**: how to combine bio with recency-weighted posts; selecting τ .
- **Retriever choice**: speed vs domain fit (hashtags, slang) and multilingual coverage.
- **Candidate depth**: N for recall; M for precision vs latency.
- **Calibration**: prevent engagement priors from swamping semantic relevance (normalization + gating).
- **Policy handling**: hard blocks vs soft penalties for safety/authenticity, potentially brand-specific.

7) Efficiency: How I Would Evaluate the System

Offline quality

- **Precision@K, Recall@K, NDCG@K** (graded relevance primary)
- **Constraint satisfaction rate**: fraction of Top- K meeting filters (e.g., min followers, geo/lang)
- **Slice metrics**: vertical/geo/lang/safety to detect regressions and distribution shift
- **Worst-query inspection**: per-stage traces (BM25 vs ANN vs CE) for targeted fixes

Latency/throughput

Track per-stage timing (and tail latency P95/P99) as a function of (N, M) :

$$T \approx T_{\text{retrieve}}(N) + T_{\text{CE}}(M)$$

Choose (N, M) to maximize NDCG under a latency budget; treat CE batching and M as the primary lever.

8) Labeling Strategy + Expense

Pragmatic approach: **weak supervision + small human validation** (teacher-student).

- Use a strong cross-encoder (teacher) to generate graded labels for (q, c) pairs.
- Sample **hard negatives** from ANN neighbors that are embedding-close but semantically wrong.
- Human-validate a smaller subset with a clear rubric (relevant / somewhat / not relevant + suitability flags).

Scale estimate: 10K–30K labeled pairs for calibration/eval; 2K–5K human-validated.

Cost estimate: if human labeling is \$0.10–\$0.30 per pair, 5K pairs costs \$500–\$1,500.

Conclusion

I would build a hybrid, multi-stage recommendation system: OpenSearch BM25 + ANN (HNSW) for fast, high-recall retrieval over 10M creators, cross-encoder reranking for precision, and a calibrated blend with engagement, preferences, and safety/authenticity signals. This architecture is robust to spam, compositional multi-intent queries, multilingual drift, and temporal topic changes, while keeping latency controlled via the (N, M) reranking budget.